PYTHON LEARNING GUIDE

Object-Oriented Programming & Inheritance

From first principles to cooperative multiple inheritance — intuition first, with worked examples, deliberate bugs, and graded exercises.

12 sections | 30+ verified examples | 30+ exercises

Every code snippet in this guide was executed and its output verified.

Contents

SECTION 1

From Functions to Objects

Why OOP exists and the problem it solves

Before classes, you already know how to bundle data: a dictionary. And you know how to bundle behavior: a function. Object-oriented programming is the observation that these two things almost always travel together, so the language should let you tie them into one unit.

Consider modeling a bank account with only dicts and functions:

```
account = {"owner": "Ada", "balance": 100}

def deposit(acc, amount):
    acc["balance"] += amount

def withdraw(acc, amount):
    if amount > acc["balance"]:
        raise ValueError("insufficient funds")
    acc["balance"] -= amount

deposit(account, 50)          # you must pass the data in every time
```

Two problems appear immediately. First, nothing stops anyone from writing `account["balance"] = -999` and skipping your rules. Second, the data and the functions that operate on it live in different places; the connection exists only in your head. A class fixes both by making the account *own* its operations.

THE CORE SHIFT: BUNDLING

```
PROCEDURAL                          OBJECT-ORIENTED
----------                          ---------------
data:  { balance: 100 }             +-------------------------+
                                    |   BankAccount object    |
funcs: deposit(acc, amt)  ------>   |  data:  balance = 100   |
       withdraw(acc, amt)           |  behavior:              |
                                    |    deposit(amt)         |
(data and behavior are separate;    |    withdraw(amt)        |
 you wire them by hand each call)   +-------------------------+
                                    (one unit; behavior lives WITH the data)
```

KEY IDEA · The one-sentence definition

A **class** is a blueprint. An **object** (or **instance**) is one thing built from that blueprint. The class says what data every instance holds and what it can do; each instance holds its own copy of that data.

OOP gives you four levers, which the rest of this guide builds up in order:

```
ENCAPSULATION  ->  bundle data + behavior, hide the internals
INHERITANCE    ->  build a new class on top of an existing one
POLYMORPHISM   ->  many types respond to the same call, each its own way
ABSTRACTION    ->  program to an interface, not a concrete implementation
```

PATTERN SIGNAL · When to reach for a class

You want a class when you have **state that outlives a single function call** and a **set of operations that all act on that same state**. A parser with a position cursor, an account with a balance, a game entity with health — all naturally classes. A one-off `max(a, b)` is not.

SECTION 2

Classes and Instances

__init__, self, and the dot-call mechanic

Here is the same bank account as a class. Read it once, then we dissect every keyword.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

acc = BankAccount("Ada", 100)
print(acc.owner, acc.balance)
print(acc.deposit(50))
print(type(acc), isinstance(acc, BankAccount))
```

OUTPUT

```
Ada 100
150
<class '__main__.BankAccount'> True
```

`__init__` is the *initializer* (people loosely call it the constructor). Python calls it automatically right after a new blank object is created, so its job is to fill that blank object with starting data. You never call it directly; writing `BankAccount("Ada", 100)` triggers it for you.

`self` is the instance being worked on, handed to every method as the first argument. It is not a keyword — it is just a parameter name (a strong convention). This is the single most important mechanic to internalize:

HOW SELF GETS FILLED

```
You write:                    Python actually runs:
----------                    --------------------
acc.deposit(50)        ==>    BankAccount.deposit(acc, 50)
                                                 ^^^
                                        acc is bound to `self`

So inside deposit:  self  IS  acc
                    self.balance  IS  acc.balance
```

KEY IDEA · The dot rewrites the call

`obj.method(args)` is sugar for `Class.method(obj, args)`. The object left of the dot becomes `self`. Every attribute you want to persist on the instance must be written through `self` (e.g. `self.balance = ...`). A bare `balance = ...` inside a method makes a throwaway local variable.

This next bug is the most common beginner mistake — forgetting `self`:

```
class Counter:
    def __init__(self):
        count = 0            # BUG: local variable, dies when __init__ returns

    def increment(self):
        self.count += 1      # AttributeError: no self.count was ever set

c = Counter()
c.increment()
```

OUTPUT

```
AttributeError: 'Counter' object has no attribute 'count'
```

WARNING ·Diagnosing 'has no attribute'

An `AttributeError: '...' object has no attribute 'x'` almost always means one of two things: (1) you wrote `x = ...` instead of `self.x = ...` in `__init__`, or (2) you never ran the code path that sets it (often a forgotten `super().__init__()` — see Section 6).

A method can also return a fresh object, and instances are independent — changing one never touches another:

```
a = BankAccount("Ada", 100)
b = BankAccount("Bob", 100)
a.deposit(50)
print(a.balance, b.balance)   # a changed, b did not
```

OUTPUT

```
150 100
```

Practice — Section 2

1. **Rectangle.** Write a class `Rectangle` with `__init__(self, width, height)`, a method `area()` and a method `perimeter()`. Create one that is 3×4 and print both values.

2. **Fix the bug.** The class below raises `AttributeError`. Find and fix the single mistake.

```
class Dog:
    def __init__(self, name):
        name = name
    def bark(self):
        return self.name + " says woof"
```

3. **Counter.** Write a `Counter` class that starts at 0, has `increment()` and `reset()`, and a `value()` method returning the current count. Show that two counters are independent.

Solutions — Section 2

1. Rectangle

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)

r = Rectangle(3, 4)
print(r.area(), r.perimeter())
```

```
OUTPUT
12 14
```

2. Fix the bug. `name = name` assigns a local variable to itself and never stores anything on the instance. It must be `self.name = name`.

```
class Dog:
    def __init__(self, name):
        self.name = name          # was: name = name
    def bark(self):
        return self.name + " says woof"
```

3. Counter

```
class Counter:
    def __init__(self):
        self.count = 0
    def increment(self):
        self.count += 1
    def reset(self):
        self.count = 0
    def value(self):
        return self.count

a, b = Counter(), Counter()
a.increment(); a.increment(); b.increment()
print(a.value(), b.value())
```

```
OUTPUT
2 1
```

SECTION 3

Instance vs Class Attributes

The shared-state trap and how to avoid it

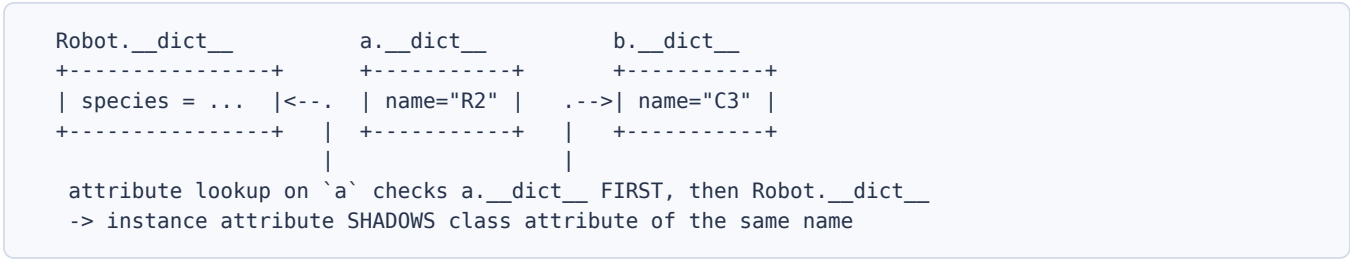
Attributes live in two different places, and confusing them is the source of a famous, hard-to-see bug. An **instance attribute** is set through `self` and belongs to one object. A **class attribute** is defined directly in the class body and is *shared by every instance*.

```
class Robot:
    species = "bot"            # class attribute: one copy, shared
    def __init__(self, name):
        self.name = name       # instance attribute: one per object

a = Robot("R2")
b = Robot("C3")
print(a.species, b.species)    # both see the shared value
Robot.species = "android"      # change it in one place...
print(a.species, b.species)    # ...everyone sees the change
```

OUTPUT

```
bot bot
android android
```

ATTRIBUTE LOOKUP ORDER



KEY IDEA ·Lookup rule

Reading `obj.x` checks the instance first, then the class, then base classes (the MRO from Section 7). **Writing** `obj.x = ...` always creates an instance attribute — it never mutates the class attribute. This asymmetry is what makes the next bug so sneaky.

The trap: a **mutable** class attribute. Because every instance shares the one object, mutating it through any instance mutates it for all.

```
class Team:
    members = []               # BUG: one list shared by ALL teams
    def __init__(self, name):
        self.name = name
    def add(self, person):
        self.members.append(person)   # mutates the SHARED list

a = Team("Alpha")
b = Team("Beta")
a.add("Ada")
b.add("Bob")
print("Alpha:", a.members)
print("Beta :", b.members)
print("same object?", a.members is b.members)
```

OUTPUT

```
Alpha: ['Ada', 'Bob']
Beta : ['Ada', 'Bob']
same object? True
```

WARNING ·Why re-binding looks like it works but appending doesn't

`self.members = [...]` (assignment) creates a fresh instance attribute and hides the bug. `self.members.append(...)` (mutation) reaches the shared class list. So the same code sometimes seems fine and sometimes leaks — depending on whether you assign or mutate.

The fix is to make the list an **instance** attribute, created fresh in `__init__`:

```python
class Team:
    def __init__(self, name):
        self.name = name
        self.members = []    # fresh list per instance
    def add(self, person):
        self.members.append(person)

a = Team("Alpha"); b = Team("Beta")
a.add("Ada")
b.add("Bob")
print(a.members, b.members, a.members is b.members)
```

OUTPUT

```
['Ada'] ['Bob'] False
```

PATTERN SIGNAL ·Rule of thumb

Use a **class attribute** only for values shared by design and rarely mutated: constants, defaults, counters of how many instances exist, configuration. Anything that is per-object state — especially lists, dicts, and sets — belongs in `__init__` as an instance attribute.

Practice — Section 3

1. **Spot the shared state.** Predict the output, then explain why.

```
class Playlist:
    songs = []
    def add(self, s): self.songs.append(s)

p1 = Playlist(); p2 = Playlist()
p1.add("A"); p2.add("B")
print(p1.songs, p2.songs)
```

2. **Instance counter.** Write a class `Widget` that tracks how many widgets have been created in a class attribute `count`, incremented in `__init__`. Create three and print `Widget.count`.

3. **Fix it.** Rewrite `Playlist` above so each playlist has its own song list.

Solutions — Section 3

1. Output is `['A', 'B'] ['A', 'B']`. `songs` is a class attribute, so both instances share the same list; both `append` calls mutate it.

2. Instance counter — a legitimate use of a mutable-free class attribute:

```
class Widget:
    count = 0
    def __init__(self):
        Widget.count += 1     # increment the CLASS attribute explicitly

Widget(); Widget(); Widget()
print(Widget.count)
```

OUTPUT

```
3
```

WARNING ·Note the explicit Widget.count

Writing `self.count += 1` would read the class value but then create an *instance* attribute holding the incremented number, leaving `Widget.count` stuck at 0. Always name the class explicitly when incrementing a shared counter.

3. Fixed Playlist

```
class Playlist:
    def __init__(self):
        self.songs = []
    def add(self, s):
        self.songs.append(s)

p1 = Playlist(); p2 = Playlist()
p1.add("A"); p2.add("B")
print(p1.songs, p2.songs)
```

OUTPUT

```
['A'] ['B']
```

SECTION 4

Methods: Instance, Class, Static

Three method types and the factory pattern

There are three kinds of methods. They differ only in *what automatic first argument* Python passes them.

THE THREE METHOD TYPES

```
KIND              first arg    receives         typical job
----              ---------    --------         -----------
instance method   self         the object       act on one object's data
@classmethod      cls          the class        alternative constructors
@staticmethod     (none)       nothing auto     related utility, no state
```

All three on one class:

```python
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    def to_fahrenheit(self):              # instance method
        return self.celsius * 9/5 + 32

    @classmethod
    def from_fahrenheit(cls, f):          # factory: builds and returns cls(...)
        return cls((f - 32) * 5/9)

    @staticmethod
    def is_freezing(celsius):             # pure helper, no self/cls
        return celsius <= 0

t = Temperature(25)
print(t.to_fahrenheit())
t2 = Temperature.from_fahrenheit(212)     # alternative way to build one
print(t2.celsius)
print(Temperature.is_freezing(-4), Temperature.is_freezing(10))
```

OUTPUT

```
77.0
100.0
True False
```

KEY IDEA · Why classmethod uses cls, not the class name

A factory written as `@classmethod` with `cls(...)` automatically does the right thing for subclasses: `cls` is whatever class the call was made on. Hard-coding `Temperature(...)` instead would break subclasses.

Watch `cls` resolve to the subclass:

```python
class KelvinTemp(Temperature):
    pass

k = KelvinTemp.from_fahrenheit(32)        # cls is KelvinTemp here
print(type(k).__name__, k.celsius)
```

OUTPUT

```
KelvinTemp 0.0
```

PATTERN SIGNAL ·Decision guide

Does the method use per-object data (`self.something`)? → **instance method**. Does it build or configure instances of the class, or need the class itself? → **@classmethod**. Is it just a logically-related function that touches neither? → **@staticmethod** (or honestly, a module-level function).

WARNING ·Common slip

Forgetting `@staticmethod` on a helper that takes no `self` means Python still injects the instance as the first argument, so `Temperature(25).is_freezing(-4)` would pass the *object* as `celsius` . The decorator is what tells Python to skip the automatic first argument.

Practice — Section 4

1. **Alternative constructor.** Add a `@classmethod` `from_string` to a `Point` class that parses `"3,4"` into `Point(3, 4)`.
2. **Classify.** For each method decide instance / class / static: (a) computes distance of a point from origin; (b) returns how many points have been created; (c) checks whether a given string like `"1,2"` is a valid coordinate format.
3. **Subclass factory.** Show that if `Point3D(Point)` inherits `from_string`, calling `Point3D.from_string("1,2")` returns a `Point`, and explain why. What would you change so it returns a `Point3D`?

Solutions — Section 4

1. from_string factory

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    @classmethod
    def from_string(cls, s):
        x, y = s.split(",")
        return cls(int(x), int(y))

p = Point.from_string("3,4")
print(p.x, p.y)
```

```
OUTPUT
3 4
```

2. Classification. (a) **instance** — needs `self.x, self.y`. (b) **class** — needs the shared counter on the class. (c) **static** — validates a string, touches no instance and no class state.

3. Because `from_string` uses `cls`, calling it on `Point3D` makes `cls = Point3D`. But `Point3D.__init__` likely needs three coordinates, so `cls(int(x), int(y))` would fail unless `Point3D` accepts two args. If `Point3D`'s constructor signature matches, it already returns a `Point3D` — no change needed. The subtle point: a well-written `cls`-based factory is inherited correctly; a factory hard-coded as `Point(...)` would always return a `Point` even from `Point3D`.

SECTION 5

Encapsulation and Properties

Underscores, name mangling, property, __slots__

Encapsulation means controlling access to an object's internals so that invariants (rules that must always hold) cannot be violated from outside. Python has no true `private` keyword; it uses convention plus one mechanism.

NAMING CONVENTIONS

```
  name        convention meaning
  ----        ------------------
  balance     public   -> free to use
  _balance    "internal, don't touch"  (a polite request; not enforced)
  __balance   name-mangled -> becomes _ClassName__balance (avoids clashes)
```

The single leading underscore is a signal to humans, enforced by nobody. The double leading underscore triggers **name mangling**: Python rewrites `__x` to `_ClassName__x` so subclasses don't accidentally clobber a base class's attribute. It is not security — you can still reach it if you spell the mangled name.

```
class Vault:
    def __init__(self):
        self.__secret = 42            # stored as _Vault__secret

v = Vault()
print(hasattr(v, "_Vault__secret"), v._Vault__secret)
try:
    print(v.__secret)                 # this name does not exist
except AttributeError as e:
    print("caught:", type(e).__name__)
```

OUTPUT

```
True 42
caught: AttributeError
```

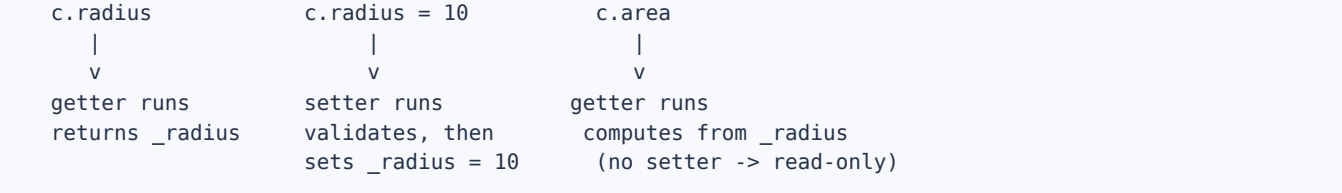
The real tool for controlled access is `@property`. It lets an attribute run code on read and write while still looking like a plain attribute to callers — so you can add validation without changing anyone's code.

```python
class Circle:
    def __init__(self, radius):
        self.radius = radius          # goes THROUGH the setter below

    @property
    def radius(self):                 # getter: obj.radius reads this
        return self._radius

    @radius.setter
    def radius(self, value):          # setter: obj.radius = v runs this
        if value < 0:
            raise ValueError("radius must be non-negative")
        self._radius = value

    @property
    def area(self):                   # computed, read-only (no setter)
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.radius, round(c.area, 2))
c.radius = 10
print(c.radius, round(c.area, 2))
try:
    c.radius = -1
except ValueError as e:
    print("caught:", e)
try:
    c.area = 100                      # no setter -> cannot assign
except AttributeError as e:
    print("caught:", type(e).__name__)
```

OUTPUT

```
5 78.54
10 314.16
caught: radius must be non-negative
caught: AttributeError
```

WHAT PROPERTY INTERCEPTS



KEY IDEA · Why property beats get_x()/set_x()

You can ship a class with a plain public attribute today, and later turn it into a property to add validation or computation — **without changing a single line of calling code**. That backward compatibility is the whole point. Start with plain attributes; reach for `property` the moment you need a rule enforced or a value computed on access.

A related memory tool is `__slots__`, which fixes the exact set of allowed attributes. It saves memory and catches typos, at the cost of losing the per-instance `__dict__`.

```
class Pt:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Pt(1, 2)
print(p.x, p.y)
try:
    p.z = 3                          # not in __slots__
except AttributeError as e:
    print("caught:", e)
```

OUTPUT

```
1 2
caught: 'Pt' object has no attribute 'z'
```

WARNING ·Property pitfall: infinite recursion

If the getter returns `self.radius` (the property name) instead of `self._radius` (the backing field), the getter calls itself forever → `RecursionError`. The property and its backing attribute must have different names — the underscore convention (`_radius`) is exactly why.

Practice — Section 5

1. **Validated temperature.** Write a class where `celsius` is a property that rejects values below −273.15, and `fahrenheit` is a computed property (getter only) derived from celsius.

2. **Spot the recursion.** Why does this raise `RecursionError`? Fix it.

```
class Account:
    def __init__(self, balance):
        self.balance = balance
    @property
    def balance(self):
        return self.balance
```

3. **Read-only id.** Give a class an `id` that is set once in `__init__` and cannot be reassigned afterward. Demonstrate that `obj.id = 5` raises.

Solutions — Section 5

1. Validated temperature

```
class Temp:
    def __init__(self, celsius):
        self.celsius = celsius
    @property
    def celsius(self):
        return self._celsius
    @celsius.setter
    def celsius(self, v):
        if v < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = v
    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

t = Temp(25)
print(t.fahrenheit)
try:
    Temp(-300)
except ValueError as e:
    print("caught:", e)
```

```
OUTPUT
77.0
caught: below absolute zero
```

2. The getter returns `self.balance`, which invokes the getter again — infinite recursion. Use a differently-named backing field:

```
class Account:
    def __init__(self, balance):
        self._balance = balance     # store in backing field
    @property
    def balance(self):
        return self._balance        # read backing field, not self.balance
```

3. Read-only id — a setter that refuses a second write:

```
class Entity:
    def __init__(self, id):
        self._id = id
    @property
    def id(self):
        return self._id
    # deliberately no setter -> assignment raises AttributeError

e = Entity(7)
print(e.id)
try:
    e.id = 5
except AttributeError as e2:
    print("caught:", type(e2).__name__)
```

OUTPUT

```
7
caught: AttributeError
```

SECTION 6

Inheritance and super()

is-a, overriding, and extending the parent

Inheritance lets a new class (the **subclass** / child) reuse and extend an existing one (the **superclass** / base / parent). The relationship you are claiming is **is-a**: a `Dog` *is an* `Animal`. If that sentence sounds wrong, inheritance is the wrong tool (see Section 9).

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f"{self.name} makes a sound"
    def describe(self):
        return f"I am {self.name}"

class Dog(Animal):                          # Dog inherits from Animal
    def __init__(self, name, breed):
        super().__init__(name)              # run Animal's __init__ first
        self.breed = breed
    def speak(self):                        # OVERRIDE Animal.speak
        return f"{self.name} barks"

d = Dog("Rex", "Lab")
print(d.speak())                            # Dog's version
print(d.describe())                         # inherited unchanged from Animal
print(d.breed)
print(isinstance(d, Animal), issubclass(Dog, Animal))
```

OUTPUT

```
Rex barks
I am Rex
Lab
True True
```

METHOD LOOKUP WALKS THE CHAIN

```
  looking up  d.describe()          looking up  d.speak()
  --------------------------          ----------------------
  1. Dog.__dict__   ? no              1. Dog.__dict__   ? YES -> use it
  2. Animal.__dict__? YES -> use it      (stops; never reaches Animal)

  This search path (Dog -> Animal -> object) is the MRO. Overriding
  works because the subclass is found FIRST.
```

KEY IDEA · super() means 'the next class in the MRO', not 'my parent'

`super().__init__(name)` calls the initializer of the next class up the chain and lets you **extend** behavior instead of replacing it. In single inheritance that next class is the parent; in multiple inheritance it is subtler (Section 7). Always prefer `super().method(...)` over naming the base class directly.

The most common inheritance bug is forgetting to call `super().__init__`, which leaves the parent's attributes unset:

```
class Cat(Animal):
    def __init__(self, name, indoor):
        self.indoor = indoor          # BUG: never ran super().__init__(name)

c = Cat("Milo", True)
try:
    print(c.speak())                  # speak() needs self.name
except AttributeError as e:
    print("caught:", e)
```

OUTPUT

```
caught: 'Cat' object has no attribute 'name'
```

WARNING ·If you define __init__ in a subclass, you are responsible for the parent's setup

Defining `__init__` in the child *replaces* the parent's, so the parent's attributes never get created unless you call `super().__init__(...)`. If the subclass adds no new fields, omit `__init__` entirely and let the parent's run automatically.

Overriding while still reusing the parent's work is the everyday pattern — call up, then add:

```
class Employee:
    def __init__(self, name, base):
        self.name, self.base = name, base
    def pay(self):
        return self.base

class Manager(Employee):
    def __init__(self, name, base, bonus):
        super().__init__(name, base)
        self.bonus = bonus
    def pay(self):
        return super().pay() + self.bonus   # reuse parent, then extend

print(Manager("Ada", 1000, 500).pay())
```

OUTPUT

```
1500
```

Practice — Section 6

1. **Vehicle hierarchy.** Write `Vehicle` with `__init__(self, wheels)` and a method `describe()` returning `"vehicle with N wheels"`. Write `Car(Vehicle)` that fixes wheels to 4 and adds a `brand`, overriding `describe()` to include the brand while reusing the parent's text via `super()`.

2. **Find the bug.** Why does `Student("Ana", 20).greet()` fail? Fix with one line.

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return "Hi, I am " + self.name
class Student(Person):
    def __init__(self, name, age):
        self.age = age
```

3. **Extend, don't replace.** Given `Logger` with `log(msg)` that returns `"[LOG] msg"`, write `TimestampLogger` whose `log` returns `"[2026] [LOG] msg"` by calling the parent's `log`.

Solutions — Section 6

1. Vehicle / Car

```
class Vehicle:
    def __init__(self, wheels):
        self.wheels = wheels
    def describe(self):
        return f"vehicle with {self.wheels} wheels"

class Car(Vehicle):
    def __init__(self, brand):
        super().__init__(4)
        self.brand = brand
    def describe(self):
        return f"{self.brand}: " + super().describe()

print(Car("Toyota").describe())
```

> OUTPUT
>
> ```
> Toyota: vehicle with 4 wheels
> ```

2. `Student.__init__` never calls `super().__init__(name)`, so `self.name` is never set. Fix:

```
class Student(Person):
    def __init__(self, name, age):
        super().__init__(name)      # add this line
        self.age = age
```

3. TimestampLogger

```
class Logger:
    def log(self, msg):
        return f"[LOG] {msg}"

class TimestampLogger(Logger):
    def log(self, msg):
        return "[2026] " + super().log(msg)

print(TimestampLogger().log("started"))
```

> OUTPUT
>
> ```
> [2026] [LOG] started
> ```

SECTION 7

MRO and the Diamond Problem

C3 linearization and cooperative super()

When a class inherits from more than one parent, Python needs a rule for which method wins and in what order `super()` walks the chain. That rule is the **Method Resolution Order (MRO)**, computed by an algorithm called **C3 linearization**. You rarely compute it by hand, but you must be able to read it.

The classic hard case is the **diamond**: `D` inherits from `B` and `C`, both of which inherit from `A`.

THE DIAMOND AND ITS MRO

```
         A            MRO of D (C3 linearization):
        / \
       B   C          D -> B -> C -> A -> object
        \ /
         D           read left-to-right, each class appears ONCE,
                     children always come before their parents
```

```
class A:
    def __init__(self):
        print("A.__init__")
        super().__init__()
    def whoami(self):
        return "A"

class B(A):
    def __init__(self):
        print("B.__init__")
        super().__init__()
    def whoami(self):
        return "B -> " + super().whoami()

class C(A):
    def __init__(self):
        print("C.__init__")
        super().__init__()
    def whoami(self):
        return "C -> " + super().whoami()

class D(B, C):
    def __init__(self):
        print("D.__init__")
        super().__init__()

print("MRO:", [k.__name__ for k in D.__mro__])
print("--- constructing D() ---")
d = D()
print("whoami:", d.whoami())
```

OUTPUT

```
MRO: ['D', 'B', 'C', 'A', 'object']
--- constructing D() ---
D.__init__
B.__init__
C.__init__
A.__init__
whoami: B -> C -> A
```

KEY IDEA · super() follows the MRO, not the class tree

Inside `B`, `super()` does **not** mean 'my parent A'. It means 'the next class after B in D's MRO', which is `C`. That is why constructing a `D` runs B, then C, then A — and A runs **exactly once**. This is the payoff of cooperative `super()`: no parent gets initialized twice.

Contrast with hard-coding parent calls, which breaks the diamond by running `A` twice and skipping `C` on the main path:

```
class A:
    def __init__(self): print("A.__init__")
class B(A):
    def __init__(self):
        print("B.__init__")
        A.__init__(self)        # BUG: names A directly instead of super()
class C(A):
    def __init__(self):
        print("C.__init__")
        A.__init__(self)        # BUG
class D(B, C):
    def __init__(self):
        print("D.__init__")
        B.__init__(self)        # BUG: bypasses the MRO
        C.__init__(self)

D()
```

OUTPUT

```
D.__init__
B.__init__
A.__init__
C.__init__
A.__init__
```

WHY DOUBLE-INIT MATTERS

```
super() version:  A.__init__ runs 1 time   (correct)
hardcoded version: A.__init__ runs 2 times  (BUG -- double init)

With real state (e.g. opening a file, incrementing a counter),
running a base __init__ twice corrupts data or leaks resources.
```

Some inheritance orders are impossible. If you ask for an order that cannot be linearized consistently, Python refuses at class-definition time:

```
class X: pass
class Y(X): pass
class Z(X, Y): pass       # X before Y, but Y is a subclass of X -> conflict
```

OUTPUT

```
TypeError: Cannot create a consistent method resolution order (MRO) for ...
```

WARNING ·Reading a TypeError about MRO

It means your base-class list contradicts the subclass relationships (you listed a base before one of its own subclasses). Fix it by ordering bases from **most-derived to least-derived**, or by rethinking the hierarchy — usually a sign you should use composition (Section 9).

Practice — Section 7

1. **Predict the MRO.** For `class D(B, C)` where `B(A)` and `C(A)`, write the MRO by hand, then verify with `D.__mro__`.
2. **Which whoami?** Given the cooperative diamond above, what does `D().whoami()` return and why does it visit C between B and A?
3. **Break it, then explain.** Create three classes that produce the MRO `TypeError`. State the contradictory constraint in one sentence.

Solutions — Section 7

1. The MRO is `D, B, C, A, object`. C3 keeps children before parents and preserves the left-to-right order of bases (`B` before `C`), merging the two `A` paths into a single `A` at the end.

2. It returns `"B -> C -> A"`. Inside `B.whoami`, `super()` is the next class in D's MRO, which is `C` (not `A`). Then `C.whoami`'s `super()` is `A`. So the chain threads B → C → A even though B and C are siblings in the tree.

3. Any inconsistent order works; the constraint violated is 'a class must appear before its own subclasses in every linearization':

```
class X: pass
class Y(X): pass
try:
    class Z(X, Y): pass
except TypeError as e:
    print("caught:", str(e)[:40])
```

```
OUTPUT
caught: Cannot create a consistent method res
```

Contradiction: listing `X` before `Y` demands X precede Y, but Y subclasses X so Y must precede X. No order satisfies both.

SECTION 8

Cooperative Multiple Inheritance

Mixins and the **kwargs init recipe

The practical use of multiple inheritance is the **mixin**: a small class that adds one capability (serialization, logging, timestamping) and is meant to be combined with others. Making mixins cooperate requires a disciplined `__init__` signature so arguments flow correctly through the whole chain.

KEY IDEA · The cooperative-init recipe

Every class in a multiple-inheritance chain should (1) accept `**kwargs`, (2) pull out only the arguments it owns, and (3) pass the rest up with `super().__init__(**kwargs)`. Use keyword-only parameters (after `*`) so there is no ambiguity about which class consumes which argument. The chain ends at `object.__init__`, which must receive no leftover arguments.

```
class Base:
    def __init__(self, **kwargs):
        super().__init__(**kwargs)          # ends the chain at object

class Serializable(Base):
    def __init__(self, *, fmt="json", **kwargs):
        self.fmt = fmt                       # consume only `fmt`
        super().__init__(**kwargs)           # pass the rest on

class Timestamped(Base):
    def __init__(self, *, ts=0, **kwargs):
        self.ts = ts
        super().__init__(**kwargs)

class Record(Serializable, Timestamped):
    def __init__(self, name, **kwargs):
        self.name = name
        super().__init__(**kwargs)

r = Record("log", fmt="xml", ts=42)
print(r.name, r.fmt, r.ts)
print([c.__name__ for c in Record.__mro__])
```

OUTPUT

```
log xml 42
['Record', 'Serializable', 'Timestamped', 'Base', 'object']
```

ARGUMENTS FLOW UP THE MRO

```
  Record("log", fmt="xml", ts=42)
     |
     v  keeps name="log", forwards {fmt:"xml", ts:42}
  Serializable   keeps fmt="xml",  forwards {ts:42}
     |
     v
  Timestamped    keeps ts=42,      forwards {}
     |
     v
  Base -> object  nothing left -> chain ends cleanly

  Each class grabs its own kwargs and passes the remainder up the MRO.
```

PATTERN SIGNAL ·How to recognize a mixin

A mixin is a class that (a) is never instantiated on its own, (b) adds behavior rather than representing a 'thing', and (c) is named with a capability-ish name like `LoggingMixin` or `JSONSerializableMixin`. It expresses **has-a-capability**, not **is-a**. List mixins *before* the main base class so their methods take precedence in the MRO.

The failure mode: a class in the chain that *doesn't* cooperate — it forgets `**kwargs` or doesn't call `super().__init__` — silently drops the rest of the chain:

```
class Loud:
    def __init__(self, volume=10):
        self.volume = volume
        # BUG: no super().__init__ -> anything after Loud in the MRO is skipped

class Timestamped:
    def __init__(self, ts=0):
        self.ts = ts

class Speaker(Loud, Timestamped):
    pass

s = Speaker()
print(hasattr(s, "volume"), hasattr(s, "ts"))   # ts never got set
```

OUTPUT

```
True False
```

WARNING ·Silent, not loud

This bug does not raise — it just leaves attributes unset, surfacing later as a distant `AttributeError`. When mixing classes, verify every one calls `super().__init__(**kwargs)`. One non-cooperative link breaks the whole chain below it.

Practice — Section 8

1. **Build a mixin.** Write a `ReprMixin` whose `__repr__` returns `ClassName(attr=val, ...)` using `self.__dict__`. Combine it with a plain `Point(x, y)` class so `repr(Point(1, 2))` works.
2. **Trace the flow.** For the cooperative `Record` example, list which class consumes each of `name`, `fmt`, `ts`, and what `object.__init__` finally receives.
3. **Fix the chain.** Rewrite `Loud` so `Speaker()` sets both `volume` and `ts`. State the two changes needed.

Solutions — Section 8

1. ReprMixin

```
class ReprMixin:
    def __repr__(self):
        pairs = ", ".join(f"{k}={v!r}" for k, v in self.__dict__.items())
        return f"{type(self).__name__}({pairs})"

class Point(ReprMixin):
    def __init__(self, x, y):
        self.x, self.y = x, y

print(repr(Point(1, 2)))
```

```
OUTPUT
Point(x=1, y=2)
```

2. `Record` consumes `name`; `Serializable` consumes `fmt`; `Timestamped` consumes `ts`; `Base` forwards an empty `**kwargs`, so `object.__init__` receives nothing — which is required, since `object.__init__` rejects extra arguments.

3. Two changes: give `Loud.__init__` a `**kwargs` parameter, and add `super().__init__(**kwargs)` so the MRO continues to `Timestamped`. (Both mixins should also accept `**kwargs` for full generality.)

```
class Loud:
    def __init__(self, volume=10, **kwargs):
        self.volume = volume
        super().__init__(**kwargs)
class Timestamped:
    def __init__(self, ts=0, **kwargs):
        self.ts = ts
        super().__init__(**kwargs)
class Speaker(Loud, Timestamped):
    pass

s = Speaker()
print(hasattr(s, "volume"), hasattr(s, "ts"))
```

```
OUTPUT
True True
```

SECTION 9

Polymorphism, Duck Typing, ABCs

One interface, many implementations

Polymorphism means the same call works on many types, each responding in its own way. You write code against a shared interface and let each object supply the specifics.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        ...
    def describe(self):                      # concrete, shared by all shapes
        return f"{type(self).__name__} with area {self.area():.2f}"

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

for s in [Rectangle(3, 4), Circle(5)]:
    print(s.describe())                      # same call, different area()
```

OUTPUT

```
Rectangle with area 12.00
Circle with area 78.54
```

Abstract Base Classes (ABCs) turn 'you must implement this' from a comment into an enforced contract. A class inheriting `ABC` with any `@abstractmethod` cannot be instantiated until every abstract method is overridden.

```
try:
    Shape()                      # abstract: has an unimplemented area()
except TypeError as e:
    print("caught:", str(e)[:55])

class Broken(Shape):
    pass                         # still no area() -> still abstract
try:
    Broken()
except TypeError as e:
    print("caught Broken:", str(e)[:40])
```

OUTPUT

```
caught: Can't instantiate abstract class Shape without an imple
caught Broken: Can't instantiate abstract class Broken
```

ABC AS A CONTRACT

```
Shape (ABC)               enforced contract:
+-- area()   @abstractmethod  -> subclass MUST implement
+-- describe()               -> subclass inherits for free

Rectangle(Shape)  implements area()  -> instantiable
Circle(Shape)     implements area()  -> instantiable
Broken(Shape)     forgot area()      -> TypeError at construction
```

KEY IDEA · ABC vs duck typing

Python also supports **duck typing**: 'if it walks like a duck and quacks like a duck, treat it as a duck.' You don't need a shared base class — you just call the method and it works if the object has it. ABCs add a safety net (fail early, at construction, with a clear message) when an interface is important enough to enforce.

Duck typing needs no common ancestor at all:

```python
class Duck:
    def quack(self): return "quack"
class Person:
    def quack(self): return "I'm quacking"

def make_it_quack(thing):
    return thing.quack()          # works for anything with .quack()

print(make_it_quack(Duck()), "|", make_it_quack(Person()))
```

OUTPUT

```
quack | I'm quacking
```

PATTERN SIGNAL · Which to choose

Reach for an **ABC** when you are designing a framework or library and want to guarantee that plugins implement a required interface — the enforced contract is worth it. Rely on plain **duck typing** for lightweight internal code where the flexibility outweighs the guardrails. Both are polymorphism; they differ only in how strictly the interface is enforced.

WARNING · The '...' body is not optional magic

An `@abstractmethod` still needs a body — `...` (Ellipsis) or `pass` or a docstring. The decorator is what makes it abstract, not an empty body. Forgetting the decorator turns it into an ordinary method with a useless body that subclasses are not required to override.

Practice — Section 9

1. **Payment interface.** Define an ABC `PaymentMethod` with abstract `pay(amount)`. Implement `CreditCard` and `PayPal`. Write a function `checkout(method, amount)` that works polymorphically over both.
2. **Enforce it.** Show that a subclass of `PaymentMethod` that forgets `pay` raises at instantiation, and name the exception type.
3. **Duck typing.** Write a function `total_area(shapes)` that sums `.area()` over any iterable of objects that have an `area()` method — no shared base class required. Test it with a mix of your own classes.

Solutions — Section 9

1. Payment interface

```
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        ...

class CreditCard(PaymentMethod):
    def pay(self, amount):
        return f"charged ${amount} to card"

class PayPal(PaymentMethod):
    def pay(self, amount):
        return f"sent ${amount} via PayPal"

def checkout(method, amount):
    return method.pay(amount)

print(checkout(CreditCard(), 50))
print(checkout(PayPal(), 75))
```

```
OUTPUT
charged $50 to card
sent $75 via PayPal
```

2. The exception is `TypeError`, raised at construction time:

```
class Cash(PaymentMethod):
    pass                          # no pay() -> abstract

try:
    Cash()
except TypeError as e:
    print(type(e).__name__, "->", str(e)[:45])
```

```
OUTPUT
TypeError -> Can't instantiate abstract class Cash wit
```

3. total_area via duck typing

```
def total_area(shapes):
    return sum(s.area() for s in shapes)

class Sq:
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2
class Tri:
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return 0.5 * self.b * self.h

print(total_area([Sq(2), Tri(3, 4)]))
```

OUTPUT

```
10.0
```

SECTION 10

Dunder Methods and Operator Overloading

Plugging into Python's built-in syntax

Dunder (double-underscore) methods let your objects plug into Python's built-in syntax: `+`, `==`, `len()`, `print()`, sorting, hashing. You are overriding what an operator *means* for your type.

SYNTAX MAPS TO DUNDER METHODS

```
  you write        Python calls
  ---------        ------------
  a + b            a.__add__(b)
  a == b           a.__eq__(b)
  a < b            a.__lt__(b)
  len(a)           a.__len__()
  str(a) / print   a.__str__()      (human-readable)
  repr(a)          a.__repr__()     (unambiguous, for developers)
  hash(a)          a.__hash__()     (needed for set/dict keys)
```

A worked example — a `Money` type that adds, compares, prints, and can live in a set:

```python
from functools import total_ordering

@total_ordering                      # fills in >, >=, <=  from __eq__ + __lt__
class Money:
    def __init__(self, cents):
        self.cents = cents
    def __repr__(self):
        return f"Money({self.cents})"          # developer view
    def __str__(self):
        return f"${self.cents/100:.2f}"        # user view
    def __eq__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        return self.cents == other.cents
    def __lt__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        return self.cents < other.cents
    def __hash__(self):
        return hash(self.cents)
    def __add__(self, other):
        return Money(self.cents + other.cents)

a, b = Money(500), Money(250)
print(repr(a), str(a))
print(a + b)
print(a == Money(500), a < b, a > b, a >= b)
print(sorted([a, b, Money(100)]))
print(len({Money(500), Money(500), Money(100)}))   # set dedups equal keys
```

OUTPUT

```
Money(500) $5.00
$7.50
True False True True
[Money(100), Money(250), Money(500)]
2
```

KEY IDEA · `__str__` vs `__repr__`

`__str__` is the friendly form shown by `print()` and `str()`. `__repr__` is the unambiguous form shown in the REPL, in containers, and by `repr()` — ideally it looks like valid code to recreate the object. If you define only one, define `__repr__`: Python falls back to it for `str()`, so you get both for the price of one.

`NotImplemented` (returned, not raised) is how you tell Python 'I don't know how to compare with that type; try the other operand's method.' It keeps comparisons with unrelated types sane instead of crashing.

WARNING · The `__eq__` / `__hash__` pact

Defining `__eq__` makes Python set `__hash__` to `None`, so your objects become **unhashable** — they can no longer be put in a set or used as dict keys. If instances should be hashable, you must also define `__hash__`, and equal objects must hash equal.

```
class Point:
    def __init__(self, x): self.x = x
    def __eq__(self, other): return self.x == other.x
    # no __hash__ defined -> Python sets it to None

try:
    {Point(1)}
except TypeError as e:
    print("caught:", str(e)[:45])
```

OUTPUT

```
caught: unhashable type: 'Point'
```

PATTERN SIGNAL · When you need which dunder

Printing debug info → `__repr__`. Value equality (dedup, comparisons) → `__eq__` + `__hash__`. Sorting → `__lt__` (plus `@total_ordering`). Arithmetic-like domain types (money, vectors) → `__add__` and friends. A container-like type → `__len__`, `__getitem__`, `__iter__`.

Practice — Section 10

1. **Vector2D.** Implement `__add__`, `__sub__`, `__mul__` (by a scalar), `__eq__`, `__repr__`, and `__abs__` (magnitude). Verify `Vector2D(3,4) + Vector2D(1,1)`, `* 2`, and `abs(...)`.
2. **Make it hashable.** Extend `Vector2D` so it can be a set element. What must be true of two equal vectors' hashes?
3. **Ordered by magnitude.** Add `__lt__` so a list of vectors sorts by magnitude, using `@total_ordering` to get the rest of the comparisons.

Solutions — Section 10

1. Vector2D

```
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, o): return Vector2D(self.x + o.x, self.y + o.y)
    def __sub__(self, o): return Vector2D(self.x - o.x, self.y - o.y)
    def __mul__(self, k): return Vector2D(self.x * k, self.y * k)
    def __eq__(self, o): return (self.x, self.y) == (o.x, o.y)
    def __repr__(self): return f"Vector2D({self.x}, {self.y})"
    def __abs__(self): return (self.x ** 2 + self.y ** 2) ** 0.5

v = Vector2D(3, 4)
print(v + Vector2D(1, 1), v * 2, abs(v))
```

```
OUTPUT
Vector2D(4, 5) Vector2D(6, 8) 5.0
```

2. Add `__hash__` based on the same fields used by `__eq__`. The rule: **equal objects must have equal hashes** (the converse need not hold). Hashing the `(x, y)` tuple satisfies this automatically:

```
    def __hash__(self):
        return hash((self.x, self.y))
```

3. Sort by magnitude with `@total_ordering` + `__lt__`:

```
from functools import total_ordering

@total_ordering
class Vector2D:
    def __init__(self, x, y): self.x, self.y = x, y
    def __abs__(self): return (self.x**2 + self.y**2) ** 0.5
    def __eq__(self, o): return (self.x, self.y) == (o.x, o.y)
    def __lt__(self, o): return abs(self) < abs(o)
    def __repr__(self): return f"Vector2D({self.x}, {self.y})"

print(sorted([Vector2D(3, 4), Vector2D(1, 1), Vector2D(0, 2)]))
```

```
OUTPUT
[Vector2D(1, 1), Vector2D(0, 2), Vector2D(3, 4)]
```

SECTION 11

Composition vs Inheritance

is-a, has-a, LSP, and the fragile base class

Inheritance is seductive because it reuses code with one line. But it also creates the tightest possible coupling: the subclass inherits *everything* the parent exposes, and depends on the parent's internals staying stable. Often you want reuse **without** that coupling — that is **composition**: hold another object and delegate to it.

TWO WAYS TO REUSE

```
INHERITANCE  (is-a)              COMPOSITION  (has-a)
-----------                      -----------
class Car(Engine): ...           class Car:
  -> a Car IS an Engine?  NO         def __init__(self):
                                         self.engine = Engine()
wrong relationship, and Car        -> a Car HAS an Engine   YES
inherits Engine's whole API        -> Car exposes only what it chooses
```

KEY IDEA · The test: is-a vs has-a

Say the relationship out loud. 'A `Dog` is an `Animal`' → inheritance. 'A `Car` has an `Engine`' → composition. 'A `Stack` is a `list`'? No — a stack *uses* a list internally but should not expose `insert`, `sort`, or indexing. Wrong is-a claims are the #1 source of fragile hierarchies.

Inheriting from `list` to build a `Stack` looks convenient but leaks the entire list API, letting callers break the stack's invariant:

```python
class Stack(list):                # BUG: a Stack is NOT a list
    def push(self, x):
        self.append(x)

s = Stack()
s.push(1); s.push(2)
s.insert(0, 99)                   # leaked list method breaks LIFO order
print("leaked API:", s)
```

OUTPUT

```
leaked API: [99, 1, 2]
```

Composition exposes only the operations you intend:

```python
class Stack:
    def __init__(self):
        self._items = []          # HAS-a list, hidden inside
    def push(self, x):
        self._items.append(x)
    def pop(self):
        return self._items.pop()
    def __len__(self):
        return len(self._items)
    def __repr__(self):
        return f"Stack({self._items})"

s = Stack()
s.push(1); s.push(2)
print(s, len(s))                  # no insert/sort/index leak
print(s.pop())
```

OUTPUT

```
Stack([1, 2]) 2
2
```

The deeper danger is the **Liskov Substitution Principle (LSP)**: a subclass must be usable anywhere its parent is, without surprising the caller. The classic violation is `Square` inheriting `Rectangle`:

```python
class Rectangle:
    def __init__(self, w, h):
        self.w, self.h = w, h
    def set_width(self, w): self.w = w
    def set_height(self, h): self.h = h
    def area(self): return self.w * self.h

class Square(Rectangle):
    def set_width(self, w):  self.w = self.h = w   # keeps sides equal...
    def set_height(self, h): self.w = self.h = h   # ...but breaks the contract

def resize_and_check(rect):
    rect.set_width(5)
    rect.set_height(4)
    return rect.area()             # a caller expects 5 * 4 = 20

print("Rectangle:", resize_and_check(Rectangle(1, 1)))
print("Square   :", resize_and_check(Square(1, 1)))
```

OUTPUT

```
Rectangle: 20
Square   : 16
```

THE SUBSTITUTION BREAKS

```
Caller's assumption: set_width then set_height are independent.
Rectangle honors it   -> area 20   (correct)
Square secretly links them -> area 16  (LSP VIOLATION)

A Square passes isinstance(x, Rectangle) but does NOT behave like one.
Mathematically a square is-a rectangle; behaviorally it is not.
```

PATTERN SIGNAL ·Decision checklist

Prefer **composition** unless you can answer yes to all: (1) is-a is literally true; (2) the subclass can be substituted for the parent everywhere without surprises (LSP); (3) you actually want the parent's whole public API; (4) the parent is designed for extension (stable, documented). If any answer is no, hold the object as a field and delegate.

WARNING ·Fragile base class

With inheritance, a change to the parent can silently break every subclass — especially if the parent calls its own overridable methods internally. Composition contains the blast radius: your class talks to the held object only through its public interface, so internal changes there don't ripple into you.

Practice — Section 11

1. **Is-a or has-a?** Classify each and justify in one clause: (a) `ElectricCar` / `Car`; (b) `Car` / `Engine`; (c) `Playlist` / `list`; (d) `SavingsAccount` / `Account`; (e) `House` / `Door`.
2. **Refactor to composition.** Rewrite `class Queue(list)` as a composition-based queue exposing only `enqueue`, `dequeue`, and `__len__`.
3. **Fix the LSP break.** Propose a design that avoids the Square/Rectangle problem entirely. (Hint: what should the common abstraction actually be?)

Solutions — Section 11

1. (a) **is-a** — an electric car is a car. (b) **has-a** — a car contains an engine. (c) **has-a** — a playlist holds a list of songs but is not a list. (d) **is-a** — a savings account is an account. (e) **has-a** — a house has doors.

2. Composition-based Queue

```
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()
    def enqueue(self, x):
        self._items.append(x)
    def dequeue(self):
        return self._items.popleft()
    def __len__(self):
        return len(self._items)

q = Queue()
q.enqueue("a"); q.enqueue("b")
print(q.dequeue(), len(q))
```

```
OUTPUT
a 1
```

3. Make both `Square` and `Rectangle` **immutable** (no setters), or give them a common abstract base `Shape` with a read-only `area()` and no shared mutable dimensions. Without setters that leak the 'width and height vary independently' assumption, the LSP conflict disappears. The lesson: LSP violations often signal that the mutable interface — not the class tree — is the real problem.

SECTION 12

Diagnostic Cheat Sheet

Symptom-to-cause and intent-to-tool reference

This section is a lookup table. When something breaks, match the symptom to the cause. When designing, match the intent to the tool.

Error → cause → fix

You see	Likely cause	Fix
`AttributeError: object has no attribute 'x'`	Wrote `x =` not `self.x =`, or forgot `super().__init__()`	Set it via `self.x`; call `super().__init__(...)` in the child
All instances share the same list/dict	Mutable value defined as a class attribute	Create it per-instance inside `__init__`
Class counter stuck at 0	`self.count += 1` made an instance attribute	Use `ClassName.count += 1`
`RecursionError` in a property	Getter returns `self.x` (itself) not `self._x`	Use a distinct backing field `_x`
A base `__init__` runs twice	Hard-coded `Parent.__init__(self)` in a diamond	Use cooperative `super().__init__()` everywhere
`TypeError: Cannot create a consistent MRO`	Base list contradicts subclass order	List bases most-derived first, or redesign
`TypeError: Can't instantiate abstract class`	A subclass didn't implement every `@abstractmethod`	Implement the missing method(s)
`TypeError: unhashable type`	Defined `__eq__` without `__hash__`	Add `__hash__` (equal objects hash equal)
Mixin attribute silently missing	A class in the chain skipped `super().__init__(**kwargs)`	Make every class cooperative
Static helper gets the object as 1st arg	Missing `@staticmethod`	Add the decorator

Intent → tool

You want to	Use
Bundle state + behavior	a class with `__init__`
Share a constant across instances	class attribute (immutable only)
Alternative constructor	`@classmethod` returning `cls(...)`
Related utility with no state	`@staticmethod` (or a module function)
Validate/compute on attribute access	`@property` + setter
Reuse a parent and extend it	inheritance + `super().method()`
Add one capability to many classes	a cooperative mixin
Guarantee an interface is implemented	ABC + `@abstractmethod`
Custom `+`, `==`, sorting, printing	dunder methods (+ `@total_ordering`)

Reuse without is-a coupling	composition (hold a field, delegate)

The five rules to never forget

```
1. self is not magic -- it's the object, passed as the first arg.
   obj.m(a)  ==  Class.m(obj, a)

2. Mutable class attributes are shared. Per-object state -> __init__.

3. super() means "next class in the MRO", not "my parent".
   Always cooperative: super().__init__(**kwargs).

4. Define __eq__  =>  also define __hash__ (or accept unhashable).
   Define __lt__  =>  add @total_ordering for the rest.

5. Say the relationship: "is-a" -> inheritance, "has-a" -> composition.
   When unsure, prefer composition.
```

KEY IDEA · How to practice from here

Re-implement three standard library-style abstractions from scratch, using only what is in this guide: an `OrderedSet` (dunders + composition), a `Cache` with a size limit (composition + properties), and a small plugin system where plugins subclass an ABC. These exercise every concept together, which is exactly what writing real libraries demands.